

Kubernetes

Елисей Ильин
DevOps-инженер



Елисей Ильин

О спикере:

- DevOps-инженер
- опыт работы в IT 6 лет



Цели занятия

- Узнать, что такое Kubernetes и как устроены его объекты
- Понять принципы работы кластеров и инструментов kind, k3s и Minikube
- Рассмотреть основные примитивы Kubernetes и их назначение
- Освоить базовые команды kubectl для работы с кластером
- Научиться управлять манифестами с помощью Kustomize
- Разобрать типичные ошибки при работе с манифестами и научиться их избегать

План занятия

- 1 Обзор Kubernetes и структура объектов
- 2 Примитивы Kubernetes
- 3 Локальные кластеры (kind, k3s, Minikube)
- 4 Работа с кластером
- 5 Управление манифестами (Kustomize)
- 6 Типичные ошибки новичков
- 7 Итоги

Обзор Kubernetes и структура объектов



Вы узнаете

- Что такое Kubernetes
- Какие типы объектов он использует
- Как устроены YAML-манифесты
- Зачем нужны namespace и labels



Kubernetes (k8s)

фреймворк для запуска и управления приложениями в среде контейнеров

Объекты

Все объекты в kubernetes могут быть представлены в виде yaml или json файлов.

Yaml — (Yet Another Markup Language) язык разметки, совместимый с json. Использует отступы и отсюда считается более читабельным

Пример YAML файла

```
name: Bob
age: 30
isEmployee: false
nullData: null
hobbies:
  - hiking
  - cooking
address:
  city: Paris
  zipcode: "75001"
  country: France
```


Основные параметры объектов

Основные параметры верхнего уровня

- `apiVersion` — версия API
- `kind` — тип объекта
- `metadata` — метаданные для идентификации объекта
- `spec` — параметры объекта
- `status` — хранит текущий статус объекта (не указывается при создании)

Пример YAML файла для k8s

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
  namespace: default
  labels:
    env: production
    version: 1.0
spec:
  selector:
    app: nginx
  ports:
    - name: http
      port: 80
      targetPort: 80
```



Namespace

пространство имён для логического разделения объектов

Namespace

Примеры использования:

- По окружению: `prod`, `dev`, `staging`
- По проектам: `frontend`, `backend`, `database`

Создание namespace:

```
kubectl create namespace prod
```

```
kubectl create namespace dev
```

Типы объектов:

- namespace — привязанные к namespace (Pod, Service, Deployment)
- cluster-level — глобальные (Node, Namespace, StorageClass)

Пример YAML файла для k8s

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
  namespace: default
  labels:
    env: production
    version: 1.0
spec:
  selector:
    app: nginx
  ports:
    - name: http
      port: 80
      targetPort: 80
```



Labels

метки для идентификации и группировки объектов

Labels

Примеры использования:

- Поиск объектов по критериям
- Связь объектов (Service → Pod через selector)
- Управление группами объектов

Поиск подов по лейблу:

```
kubectl get pods -l app=nginx
kubectl get pods -l env=prod
kubectl get pods -l
app=nginx,env=prod
```

Создание пода с лейблами:

```
metadata:
  labels:
    app: nginx
    env: prod
    version: v1
```

Удаление всех подов с определённым лейблом:

```
kubectl delete pods -l
env=dev
```

Демонстрация работы

1. Создание namespace: ``kubectl create ns demo``
2. Применение манифеста с labels
3. Поиск объектов: ``kubectl get pods -l app=myapp``
4. Удаление по label: ``kubectl delete pods -l env=test``

Итоги

- Kubernetes — это фреймворк для управления контейнеризованными приложениями
- Объекты описываются в YAML
- namespace разделяет ресурсы
- labels помогают группировать и фильтровать объекты

Примитивы Kubernetes



Вы узнаете

- Какие базовые сущности управляют приложениями в Kubernetes: Pod, ReplicaSet, Deployment, Service, Ingress, ConfigMap, Secret и другие

Pod

Pod — минимальная единица рабочей нагрузки.

В большинстве случаев:

pod = контейнер

Возможно создание нескольких контейнеров в рамках одного пода

```
#Пример Pod
apiVersion: v1
kind: Pod
metadata:
  name: static-web
  labels:
    role: myrole
spec:
  containers:
    - name: web
      image: nginx
      ports:
        - name: web
          containerPort: 80
          protocol: TCP
```

ReplicaSet

Задаёт желаемую конфигурацию репликации и шаблон пода.

Kubernetes будет стараться поддерживать это состояние и в случае удаления или остановки пода

```
#Пример ReplicaSet
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: redis
  labels:
    tier: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      tier: redis
  template:
    metadata:
      labels:
        tier: redis
    spec:
      containers:
        - name: redis
          image: redis
```

Job

Разовый запуск пода. Обычно используется для бутстрапа или миграций

```
#Пример Job
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  template:
    spec:
      containers:
        - name: pi
          image: yii
          command: yii migrate
      restartPolicy: Never
      backoffLimit: 4
```

CronJob

Запуск Job по расписанию

```
#Пример Job
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - date; echo Hello
```

Deployment

Задаёт шаблон replicaset и занимается выкаткой приложений. При изменении объекта создаётся новый replicaset, и одновременно со старым они начинают апскейлиться и даунскейлиться соответственно

```
#Пример Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

DaemonSet

Ведёт себя аналогично deployment. Но вместо количества реплик daemonset поднимает по 1 поду на каждой ноде.

Используется для приложений, которые необходимо держать на каждой ноде, например, node-exporter и т. п.

```
#Пример DaemonSet
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
spec:
  selector:
    matchLabels:
      name: fluent-bit
  template:
    metadata:
      labels:
        name: fluent-bit
    spec:
      containers:
        - name: fluent-bit
          image: fluent-bit:1.6
```

StatefulSet

Используется для деплоя кластерных приложений. Каждый под в нём имеет фиксированное имя, на которое можно сослаться в конфиге

```
#Пример StatefulSet
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "nginx"
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
              name: web
```


ConfigMap

Используется для хранения конфигурации. Можно хранить как переменные окружения, так и файлы конфига целиком

```
#Пример ConfigMap
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-conf
data:
  config.yaml: |
    db_host: mysql
    db_user: root
```

Secret

Используется для хранения любой чувствительной информации (пароли, ключи и т. п.). В etcd хранится в зашифрованном виде

Типы:

- Opaque
- kubernetes.io/service-account-token
- kubernetes.io/dockercfg
- kubernetes.io/dockerconfigjson
- kubernetes.io/basic-auth
- kubernetes.io/ssh-auth
- kubernetes.io/tls
- bootstrap.kubernetes.io/token

```
#Пример Secret
apiVersion: v1
kind: Secret
metadata:
  name: secret-basic-auth
type:
  kubernetes.io/basic-auth
stringData:
  username: admin
  password: t0p-Secret
```

Service

Используется для направления трафика на под и балансировки в случае, если подов несколько.

Типы Service:

- ClusterIP (по умолчанию) — доступ только внутри кластера
- NodePort — открывает порт на каждой ноде
- LoadBalancer — создаёт внешний балансировщик (в облаке)

Имя сервиса используется в качестве DNS внутри кластера

```
#Пример Service
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP # по
умолчанию, можно не указывать
  selector:
    app: MyApp
  ports:
    - protocol: TCP
      port: 80 # порт
сервиса
      targetPort: 9376 # порт в
контейнере
```

Ingress

Обрабатывается Ingress контроллером, который реализует балансировку и маршрутизацию.

Популярные контроллеры:

- nginx-ingress - самый распространенный
- traefik - современный с поддержкой автоматического SSL

Основные параметры:

- path - путь URL для маршрутизации
- backend - указывает на Service, куда направлять трафик

```
#Пример Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
spec:
  rules:
    - host: myapp.example.com #
      опционально
      http:
        paths:
          - path: /testpath
            pathType: Prefix
            backend:
              service:
                name: test # имя
                Service
                port:
                  number: 80 #
                  порт Service
```

Демонстрация работы

1. Создание Pod: ``kubectl run test --image=nginx``
2. Создание Deployment из манифеста
3. Показ ReplicaSet: ``kubectl get rs``
4. Создание Service и показ DNS
5. Демонстрация работы ConfigMap и Secret



Итоги

- Kubernetes использует набор примитивов для управления жизненным циклом приложений — от запуска контейнеров до маршрутизации и конфигурации

Локальные кластеры

kind, k3s, Minikube



Вы узнаете

- Как развернуть и протестировать локальный кластер с помощью kind, k3s, Minikube
- Чем локальные кластеры отличаются по возможностям

Kind

Запускает Kubernetes кластер в Docker контейнерах.

Преимущества:

- Очень быстрый старт (<1 минуты)
- Минимальное потребление ресурсов
- Поддержка мультинодовых кластеров
- Идеально для тестирования

Kind

Установка

```
# Linux
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind

# macOS
brew install kind
```

Запуск кластера

```
kind create cluster --name my-cluster
```

Проверка

```
kubectl cluster-info --context kind-my-cluster
kubectl get nodes
```

Kind

Мультинодовый кластер (конфигурация):

```
# kind-config.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker

kind create cluster --config kind-config.yaml
```

k3s (Lightweight Kubernetes)

Облегчённый Kubernetes от Rancher. Один бинарный файл, подходит для edge и IoT.

Преимущества:

- Очень Полнофункциональный Kubernetes
- Малое потребление ресурсов (512MB RAM)
- Встроенный LoadBalancer и Storage
- Готов к продакшену

k3s (Lightweight Kubernetes)

Установка

```
curl -sfL https://get.k3s.io | sh -
```

Проверка

```
sudo k3s kubectl get nodes
```

Использование обычного kubectl

```
# Скопировать конфиг
mkdir ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $USER ~/.kube/config
```

```
# Теперь работает обычный kubectl
kubectl get nodes
```

k3s (Lightweight Kubernetes)

k3d (k3s в Docker):

```
# Установка k3d
```

```
curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
```

```
# Создание кластера
```

```
k3d cluster create mycluster
```

Minikube

Традиционный инструмент, подходит для обучения с GUI.

Преимущества:

- Нужен Dashboard с GUI
- Работа с различными драйверами виртуализации
- Обучение новичков

Minikube

Установка

```
# Linux
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# macOS
brew install minikube
```

Запуск

```
minikube start --driver=docker
```

Dashboard

```
minikube dashboard
```


Сравнение инструментов

Критерий	kind	k3s/k3d	Minikube
Скорость запуска	Очень быстро	Быстро	Медленно
Потребление RAM	Низкое (1-2GB)	Очень низкое (512MB)	Среднее (2-4GB)
Мультинодовость	Да	Да (k3d)	Ограничено
CI/CD	Отлично	Хорошо	Не подходит
Продакшен-подобность	Средняя	Высокая	Низкая
GUI Dashboard	Нет	Нет (есть Rancher)	Да
Сложность	Простой	Средняя	Простой

Рекомендации

- 1 Для разработки: kind
- 2 Для тестирования продакшн-сценариев: k3s
- 3 Для обучения с GUI: Minikube (опционально)

Демонстрация работы

1. Запуск kind: ``kind create cluster``
2. Проверка: ``kubectl get nodes``
3. Деплой простого приложения
4. Удаление кластера: ``kind delete cluster``
5. (Опционально) Показ k3d или Minikube



Итоги

- Используете kind для быстрой разработки
- Работаете с k3s/k3d для продакшн-подобных сценариев
- Minikube — для обучения и экспериментов
- Проверяете работоспособность через ``kubectl get nodes``

Работа с кластером



Вы узнаете

- Как использовать kubectl для управления приложениями

Работа с кластером

Kubectl — основная утилита для работы с k8s кластером.

Использует конфигурацию, расположенную в файле `~/.kube/config`

Переопределить расположение этого файла можно с помощью переменной `KUBECONFIG`.

k3s имеет встроенный kubectl и может использоваться как:

```
k3s kubectl
```

Отдельно эту утилиту можно установить по инструкции:

<https://kubernetes.io/ru/docs/tasks/tools/install-kubectl/>

Работа с кластером: синтаксис

Общий синтаксис:

```
kubectl действие объект флаги
```

Пример

```
kubectl get pods -n kube-system
```


Работа с кластером: синтаксис

Основные действия
get
create
edit
delete
describe
apply

Объекты
Pods
deploy
svc
secrets

Флаги могут различаться для различных команд.

Из основных необходимо указывать namespace:

- `-n``
- `--all-namespaces``

Работа с кластером: деплой

Деплоим с помощью:

```
kubectl apply -f nginx.yaml
```

Смотрим за статусом:

```
kubectl get po
```

Смотрим подробный статус пода:

```
kubectl describe po имя_пода
```

```
#Создаем файл: nginx.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

Полный цикл управления приложением

1 Деплой приложения

- `kubectl apply -f nginx.yaml`

2 Проверка статуса

- `kubectl get pods`
- `kubectl get deploy`
- `kubectl get svc`

3 Подробная информация

- `kubectl describe pod имя_пода`
- `kubectl describe deploy nginx-deployment`

Полный цикл управления приложением

4 Просмотр логов

- `kubectl logs имя_пода`
- `kubectl logs --tail 100 имя_пода`
- `kubectl logs -f имя_пода` # следить за логами в реальном времени

5 Выполнение команд внутри пода

- `kubectl exec -it имя_пода -- bash`
- `kubectl exec имя_пода -- ls /app`
- `kubectl exec имя_пода -- env`

Полный цикл управления приложением

6 Обновление приложения

- # Изменить nginx.yaml (например, версию образа)
- `kubectl apply -f nginx.yaml`
- `kubectl rollout status deployment/nginx-deployment`

7 Удаление

- `kubectl delete -f nginx.yaml` # или
- `kubectl delete deployment nginx-deployment`

Демонстрация работы

Полный деплой на живом кластере:

1. Создание всех манифестов
2. Применение их последовательно
3. Показ ``kubectl get all -n prod``
4. Вход в pod: ``kubectl exec -it ...``
5. Показ логов: ``kubectl logs -f ...``



Итоги

- Деплоите: ``kubectl apply``
- Проверяете статус: ``get``, ``describe``
- Смотрите логи: ``logs``
- Выполняете команды: ``exec``
- Обновляете приложения с rollout

Управление манифестами (Kustomize)



Вы узнаете

- Как использовать Kustomize для настройки окружений без дублирования манифестов и изменения базовых файлов



Kustomize — встроенный в kubectl инструмент для управления и кастомизации Kubernetes манифестов без использования шаблонов

Управление манифестами с Kustomize

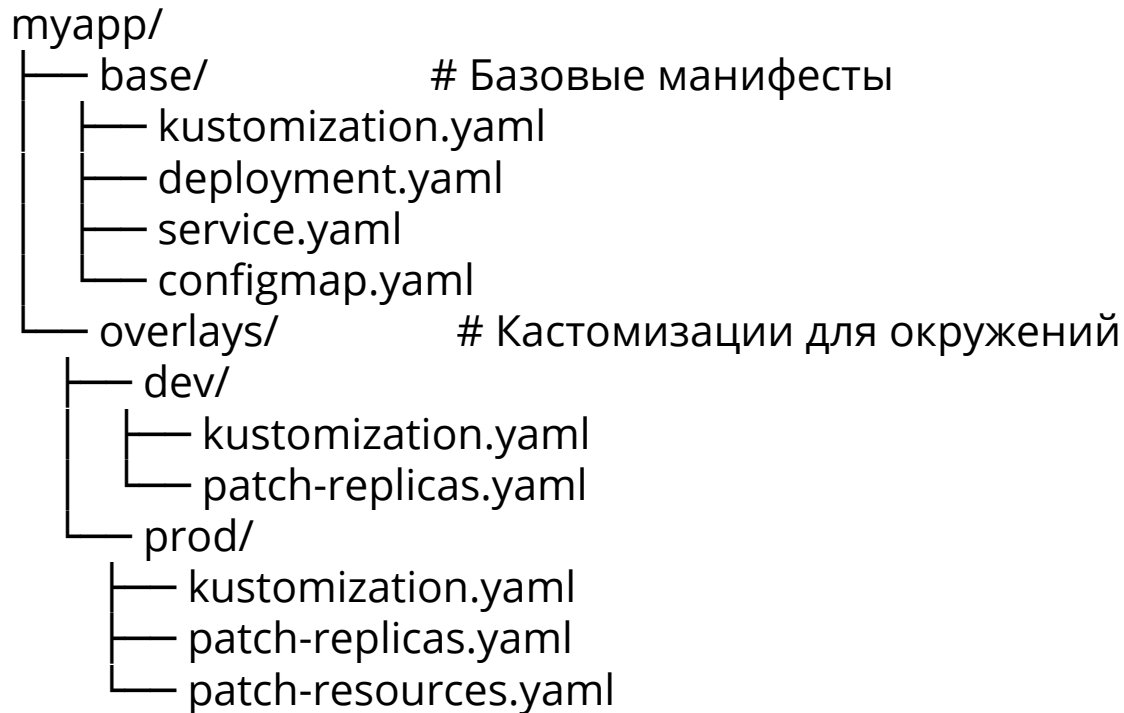
Проблемы:

- Дублирование манифестов для разных окружений (dev, prod)
- Сложность поддержки множества похожих файлов
- Необходимость изменять базовые манифесты

Решения:

- Базовые манифесты + оверлеи (overlays) для окружений
- Патчи и трансформации
- Встроен в kubectl (не нужно ставить отдельно)

Структура проекта с Kustomize



Преимущества Kustomize

- Без шаблонов — работа с чистым YAML
- DRY принцип — нет дублирования кода
- Встроен в kubectl — не нужны дополнительные инструменты
- Декларативность — все изменения в Git
- Простота — легко понять и поддерживать

Управление манифестами с Kustomize

Когда использовать:

- Несколько окружений (dev, staging, prod)
- Разные конфигурации для одного приложения
- CI/CD пайплайны

Альтернативы:

- Helm (если нужны сложные шаблоны)
- Простые манифесты (для очень простых проектов)

Демонстрация работы

Создание структуры Kustomize:

1. ``kubectl kustomize overlays/dev/`` - просмотр
2. ``kubectl apply -k overlays/dev/`` - применение
3. ``kubectl get all -n dev`` - проверка
4. Изменение replicas в prod и демонстрация diff



Итоги

- Создаёте структуру base + overlays
- Применяете кастомизации для разных окружений
- Избегаете дублирования кода
- Используете встроенный ``kubectl kustomize``

Типичные ошибки новичков



Вы узнаете

- Какие ошибки чаще всего встречаются при работе с YAML, namespace, селекторами, портами и API-версиями
- Как избежать типичных ошибок

Отступы в YAML

Ошибка:

```
spec:
  containers:
    - name: nginx
      image: nginx
      ports: # ←
Неправильный отступ!
        - containerPort:
90
```

Правильно:

```
spec:
  containers:
    - name: nginx
      image: nginx
      ports: # ←
Правильный отступ
        -
      containerPort: 80
```

Как избежать: Используйте IDE с поддержкой YAML
(VSCode + YAML extension)

Забыли указать namespace

Ошибка:

```
kubectl apply -f  
deployment.yaml  
# Создается в default  
namespace!
```

Правильно:

```
kubectl apply -f deployment.yaml  
-n my-namespace  
# Или указать в манифесте  
  
metadata:  
  name: myapp  
  namespace: prod # ← Явно  
указать
```

Как избежать: Всегда указывайте namespace в манифестах

Селекторы не совпадают с labels

Ошибка:

```
# Service
spec:
  selector:
    app: nginx # ← Ищет
app=nginx

# Deployment
template:
  metadata:
    labels:
      application: nginx # ← Не
совпадает!
```

Правильно:

```
# Оба должны совпадать
spec:
  selector:
    app: nginx
template:
  metadata:
    labels:
      app: nginx
```

Симптомы: Service не находит поды, трафик не доходит

Неправильные порты в Service

Ошибка:

```
# Service
ports:
  - port: 80
    targetPort: 8080 # Указан
неправильный порт

# Pod
containers:
  - containerPort: 80 # Слушает
на 80, а не 8080!
```

Правильно:

```
# Service
ports:
  - port: 80
    targetPort: 80 #
Совпадает с containerPort

# Pod
containers:
  - containerPort: 80
```

Устаревшие API версии

Ошибка:

```
apiVersion: extensions/v1beta1
# УСТАРЕЛО!
kind: Ingress
```

Правильно:

```
apiVersion:
networking.k8s.io/v1
# Актуально
kind: Ingress
```

Проверить актуальность: `kubectl api-resources | grep ingress`

Забыли про лимиты ресурсов

Ошибка:

```
containers:
  - name: app
    image: myapp
    # Нет requests/limits!
```

Правильно:

```
containers:
  - name: app
    image: myapp
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        cpu: 200m
        memory: 256Mi
```

Последствия: Pod может съесть все ресурсы ноды

ImagePullPolicy по умолчанию

Проблема:

```
containers:
  - name: app
    image: myapp:latest # ←
latest всегда пулится заново
```

В dev окружении: Может быть полезно

В prod окружении: Опасно! Нестабильность

Правильно для prod:

```
containers:
  - name: app
    image: myapp:v1.2.3 # ←
Конкретная версия
    imagePullPolicy:
      IfNotPresent
```

Exec в контейнер без проверки ГОТОВНОСТИ

Ошибка:

```
kubectl exec -it mypod bash  
# Error: container not ready
```

Правильно:

```
# Сначала проверить статус  
kubectl get pod mypod  
kubectl describe pod mypod  
  
# Потом exec  
kubectl exec -it mypod -- bash
```

Удаление без проверки

Ошибка:

```
kubectl delete pods -l app=myapp  
# Удалили все поды в default  
namespace!
```

Безопаснее:

```
# 1. Сначала проверить  
kubectl get pods -l app=myapp  
-n prod  
  
# 2. Потом удалить с явным  
namespace  
kubectl delete pods -l  
app=myapp -n prod
```

Игнорирование логов и events

Ошибка:

Pod не запускается, но новичок не смотрит логи

Правильный подход:

```
# 1. Статус
kubectl get pods

# 2. События
kubectl describe pod
mypod

# 3. Логи
kubectl logs mypod

# 4. Логи предыдущего
контейнера (если упал)
kubectl logs mypod --
previous
```

Чек-лист для проверки манифеста

- Проверил отступы YAML
- Указал namespace
- Селекторы совпадают с labels
- Порты настроены правильно (port, targetPort, containerPort)
- Использую актуальные API версии
- Указал requests/limits для ресурсов
- Использую конкретные версии образов (не latest)
- Добавил проверки готовности (readinessProbe, livenessProbe)

Демонстрация работы

1. Манифест с неправильным selector
2. ``kubectl apply -f wrong.yaml``
3. ``kubectl get pods`` - поды не появляются
4. ``kubectl describe deployment`` - видна ошибка
5. Исправление ошибки



Итоги

- Правильно оформляйте YAML (отступы)
- Проверяйте соответствие селекторов и labels
- Используйте актуальные API версии
- Указывайте resource limits
- Версионизируйте образы (не latest)

Итоги



Итоги занятия

- Kubernetes используется для оркестрации контейнеров и управления приложениями
- Работа строится на взаимодействии объектов, описанных в YAML-манифестах
- Основные примитивы обеспечивают масштабирование, конфигурацию и маршрутизацию
- Kubectl и Kustomize позволяют автоматизировать управление и поддерживать инфраструктуру в разных окружениях
- Знание типичных ошибок помогает сократить время настройки и избежать отказов в кластере

Kubernetes

Елисей Ильин
DevOps-инженер

